An Application - Citation Networks

EM1421 - Social media and web analytics

{Slides courtesy of Prof. Fabiana Zollo}

Prof. Pierluigi Zama Ramirez

Ca' Foscari University of Venice



Citation Networks

- Consider looking through a repository of articles published in African Studies Review
- **Question**: Which articles have been influential in the field of African Studies?

Citation Networks

- A common metric used in academia is the number of citations made to a given article
- Using citations as a measure of influence is reasonable, but simple counts only tell part of the story.

Citation Networks

For example:

- Uvin's article *Prejudice, crisis, and genocide in Rwanda*: 97 citations
- Maddox's article *Gender and Famine in Central Tanzania: 1916-1961*: 18 citations
- Clearly Uvin's article had a wider reach, but both articles might be equally central within their own sub-fields
- Maybe more scholars are interested in Rwanda or genocide than famine or Tanzania
- Maybe the 18 citations to Maddox's article are of a higher average quality compared to the 97 to Uvin's

Citation Networks

- A **citation network** is a graph where the vertices are publications and edges represent the relationship that one publication was cited by the other
- Using this **more complex graphical representation** may help to discern these, and many more, nuances

Citation Networks

- We will now work with a citation network built from US Supreme Court case opinions
- We will build a graph object of court citations starting from our data
- The citation data comes from Supreme Court Citation Network Data
- The data is a two column dataset containing about 216,000 citations between majority opinions written by the US Supreme Court
  - The first column gives the `id` of the case which cited the case in the second column
  - This is our **edge list** as each row represents an edge in the graph

Citation Networks

```
1 allCounts <- as.matrix(read.csv("ac.csv", as.is=TRUE))
2 head(allCounts)
```

```
     to         from
[1,] "4US6"     "3US320"
[2,] "4US6"     "4US1"
[3,] "6US280"   "1US393"
[4,] "6US280"   "1US53"
[5,] "6US280"   "3US133"
[6,] "14US179"  "5US321"
```

Citation Networks

- Constructing a graph object in R from an edge list involves a call to a single igraph function
- Vertices are inferred as the set of all vertices in the edge list
- In our case it will be simpler and just as interesting to have an undirected graph

```
1 library(igraph)
2 G <- graph.edgelist(allCounts, directed=FALSE)
```

Citation Networks

```
1 G
```

```
IGRAPH 3bf7eb2 UN-- 25417 216738 --
+ attr: name (v/c)
+ edges from 3bf7eb2 (vertex names):
 [1] 4US6   --3US320  4US6   --4US1    6US280 --
1US393  6US280 --1US53
 [5] 6US280 --3US133  14US179--5US321  14US179--
9US262  15US45 --3US384
 [9] 15US45 --4US436  15US45 --4US441  15US45 --
8US421  15US227--13US388
[13] 15US227--2US36   15US290--7US220  9US262 --
15US290 15US369--11US504
[17] 15US369--11US577 15US369--5US137  15US369--
9US115  15US396--1US371
[21] 15US396--4US450  15US396--6US358  15US396--
7US73   16US1  --11US603
[25] 16US1  --8US209  16US1  --8US321  16US101--
10US8   16US101--11US69
```

Citation Networks

- The entire citation graph has over 25,000 nodes
- It is possible to work with a graph of this size R, but it will be far easier for us to take a subset of the graph to work with instead
- If 100 nodes were randomly sampled from this graph, we would likely be left with a sparse and uninteresting graph
- We can instead pick out a subset of cases that deal with similar legal topics
- To do this, we will use the Supreme Court Database from the University of Washington-St. Louis to assign thematic categorization to all of the cases in our dataset

Citation Networks

The data consists of a two-column table mapping each court case to one or more "issue" codes (which can be looked up in an accompanying code book):

```
1 themes <- read.csv("themes.csv", as.is=TRUE)
2 head(themes)
```

```
     usid issue
1  329US1 80180
2 329US14 10500
3 329US29 80250
4 329US40 20150
5 329US64 80060
6 329US69 80100
```

Citation Networks

- With this metadata, it is now possible to restrict the graph `G` to only cases dealing with a few closely related issues
- We will choose to use the topics 20040 and 20050
- The `igraph` library makes this easy with the `induced.subgraph` function
  - This takes a set of vertex names and returns a graph with only the edges with endpoints in the specified set

Citation Networks

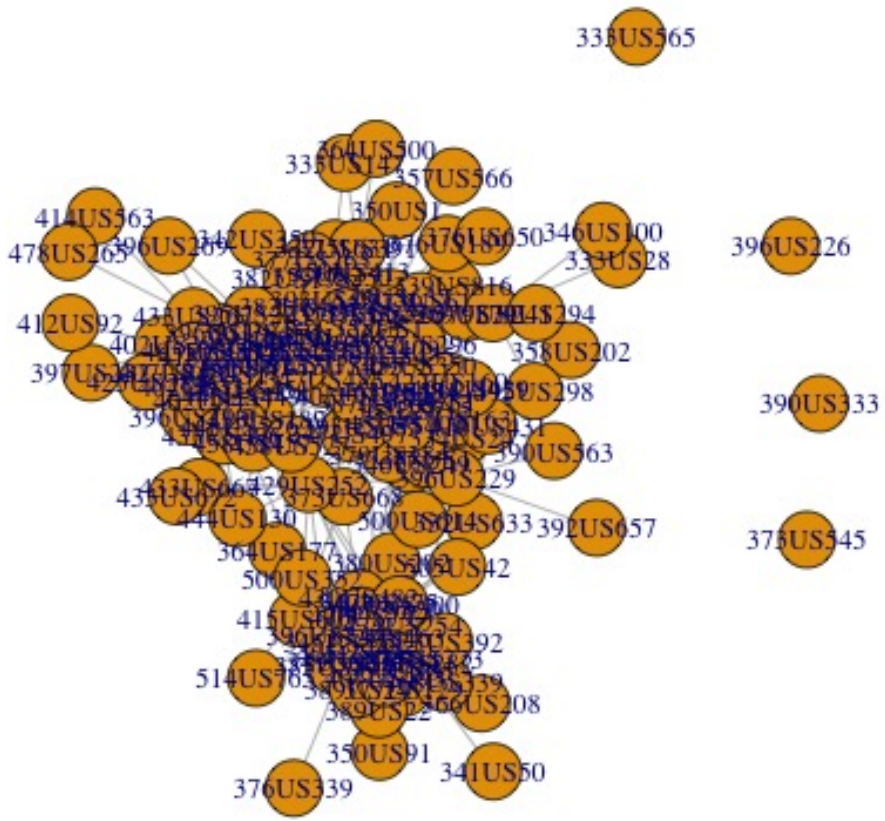
```
matchingCases <- which(themes$issue %in% c(20040, 20050))
matchingCases_ids <- themes$usid[matchingCases]
H <- induced.subgraph(G, matchingCases_ids)
H
```

```
IGRAPH c927f89 UN-- 132 729 --
+ attr: name (v/c)
+ edges from c927f89 (vertex names):
 [1] 332US463--332US633 333US147--332US631 332US633--
334US1   334US1  --334US24
 [5] 333US147--339US629 332US631--339US629 334US1  --
339US629 332US631--339US637
 [9] 334US1  --339US637 339US629--339US637 339US629--
339US637 334US1  --339US816
[13] 339US637--339US816 339US282--341US50  332US463--
345US559 339US282--345US559
[17] 333US28 --346US100 334US1  --346US249 334US24 --
346US249 339US629--346US249
[21] 339US282--347US475 332US631--347US483 339US629--
```

Citation Networks

Our graph now has only 132 vertices and is certainly small enough to easily plot:

```
1  plot.igraph(H)
```



Citation Networks

We can improve the visualization running the Fruchterman-Reingold algorithm:

```
1 set.seed(1)
2 lout <- layout.fruchterman.reingold(H)
```

- The algorithm is **nondeterministic**, that is the result depends on the random starting points of the vertices
- To ensure its reproducibility, we call the function `set.seed` just prior to running the algorithm

Citation Networks

The algorithm returns a two-dimensional matrix of coordinates:

```
1 head(lout)
```

```
            [,1]       [,2]
[1,]  4.2025621 -4.414613
[2,]  4.7158113 -1.313538
[3,]  5.2694318  5.522338
[4,] -0.4505597  5.185439
[5,] -0.3755915  3.103994
[6,]  2.6339328  0.640716
```

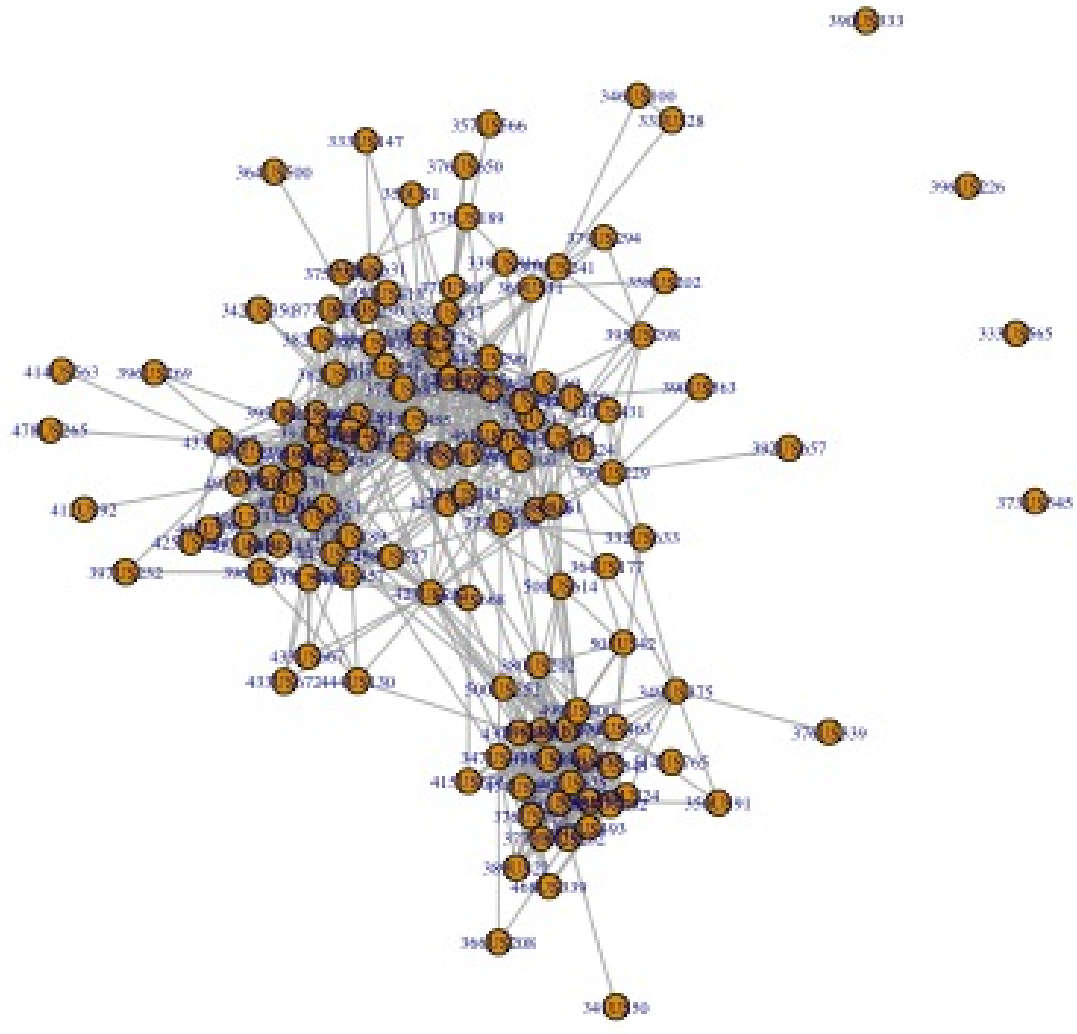
(It's in the same format we created by hand for the British Royal Family's family tree)

Citation Networks

We can now use the layout matrix to plot the citation network of our subgraph `H`:

```
plot.igraph(H, layout=lout, vertex.size=5,
            vertex.label.cex=0.5)
```

Citation Networks

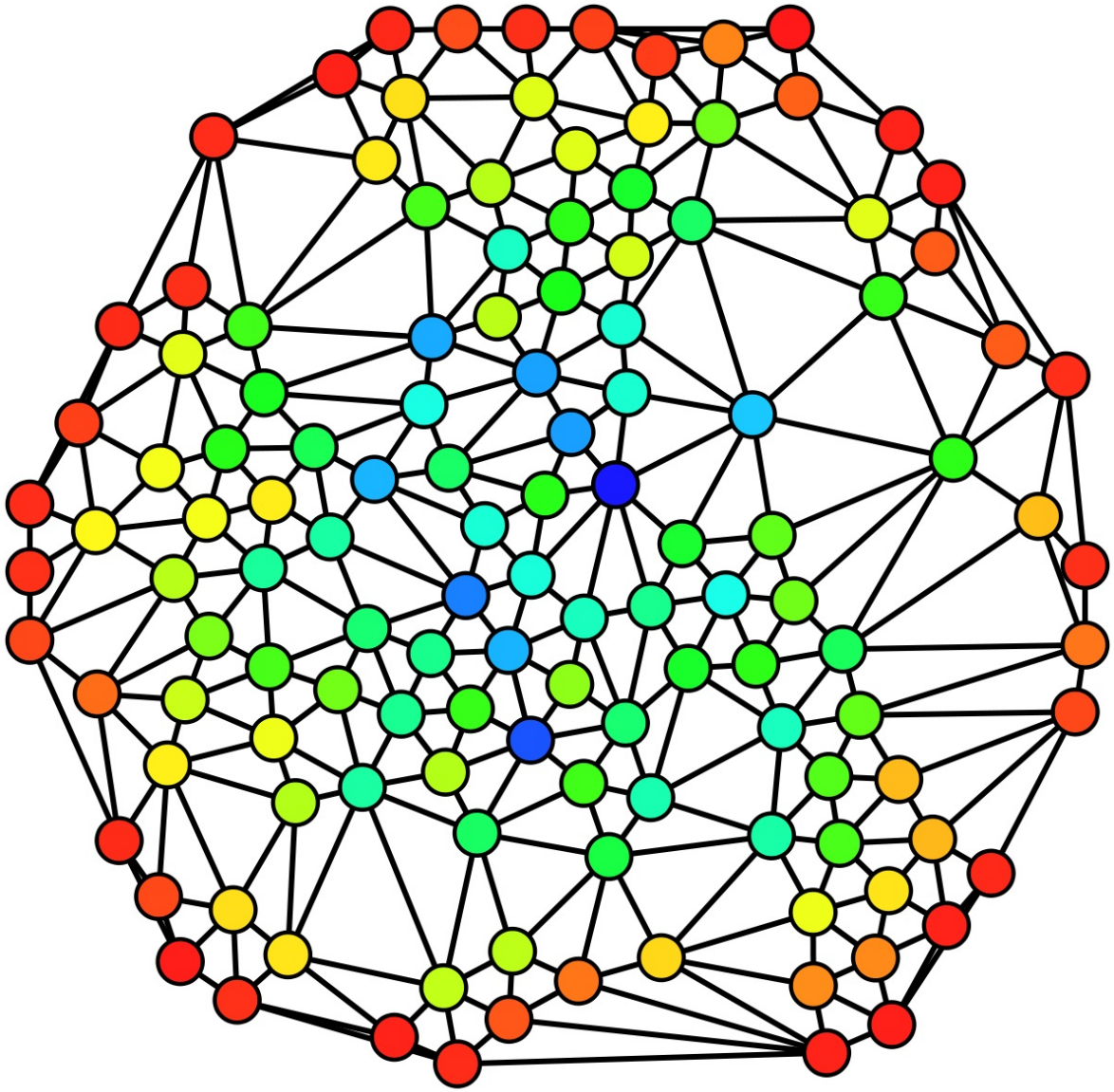


Citation Networks

To export the in high quality:

```
pdf("citation_graph.pdf",10,10)
  plot.igraph(H, layout=lout, vertex.size=10,
              vertex.label.cex=.5)
dev.off()
```

Centrality Measures



Graph Centrality

- **Centrality** is a measurement of how important a vertex is in the graph
- Examples:
  - To identify the most influential people in a social network or the most vulnerable targets in a computer network
  - The *PageRank* algorithm used by Google to order search results is a particular example of a centrality measure over the graph of websites
- In our example of a citation network, centrality can help identify the most influential opinions to a particular area of case law

Graph Centrality

- The simplest centrality measure is **degree centrality**
- Vertices are ranked only on the number of edges to which they are connected
- In citation networks, degree centrality is analogous to just counting the number of citations a document has

Graph Centrality

- A refinement is the **eigenvector centrality**
  - It assigns a higher weight to a vertex for being connected to vertices which are themselves important
  - Formally, it assigns a weight to every vertex such that the weight of a particular vertex is proportional to the sum of the weights of its neighbors

Graph Centrality

- Calculating eigenvector centrality for our subgraph `H` in R can be done using the `evcent` function
- A list with many components is returned, though only the element named "vector" will be useful for us
- This element is a named vector relating each node a numerical score between 0 and 1

```
1 eigenCent <- evcent(H)$vector
2 sort(eigenCent,decreasing=TRUE)[1:10]
```

```
 347US483  391US430  349US294    402US1  443US449  413US189
1.0000000 0.8933058 0.8011224 0.7770846 0.7493045 0.7275929
 407US451   515US70
0.6298709 0.6009171
```

Graph Centrality

- Sorting the output shows the top ten cases by their eigenvector centrality values
- The top score goes to 347US483, *Brown v. Board of Education*
- Indeed, it is largely considered the most pivotal case in school desegregation, in which the court unanimously argued that racially segregated public schools were unconstitutional

```
1 eigenCent <- evcent(H)$vector
2 sort(eigenCent,decreasing=TRUE)[1:10]
```

```
 347US483  391US430  349US294    402US1  443US449  413US189
1.0000000 0.8933058 0.8011224 0.7770846 0.7493045 0.7275929
 407US451   515US70
0.6298709 0.6009171
```

Graph Centrality

- Centrality measures are useful for identifying a list of the most influential vertices
- They can also be useful in adding context to a visualization of the entire graph by coloring vertices based on their centrality scores
- Eigenvalue centrality scores are known to typically decay very rapidly
- In our example over 90% of the vertices have a score of less than 0.5 and the median score is only 0.11

Graph Centrality

It make sense to convert our centrality scores into quantized buckets:

```
# calculating 30 values (bins)
# using the quantile function
# in each bin there is the same number
# (frequency) of scores
bins = unique(quantile(eigenCent, seq(0,1,length.out=30)))
```

Graph Centrality

```
1 bins
```

```
 [1] 0.00000000000000005680953 0.00248938730206682730383 7
 [3] 0.00801026604062385870486 0.01577705647805137390005 1
 [5] 0.02309542726840894955042 0.02745059179040126168258 6
 [7] 0.03460183430482804273209 0.04294666398341585866749 2
 [9] 0.05689958682807324391594 0.06301278565466394976368 7
[11] 0.06537612308835286101249 0.07013773716622351273031 2
[13] 0.07626779367747785198794 0.09822559025878459249003 8
[15] 0.10882688430790899575662 0.11947450016807414918673 7
[17] 0.12979812319156427968813 0.13882537786733914475512 1
[19] 0.18017548045911105369931 0.19415238799205844499873 3
[21] 0.21533838974489608042439 0.25237870005510104620682 9
[23] 0.30132821435061263315446 9 0.35505953455301914045705 3
[25] 0.39222039072406816861970 9 0.42436327932847639488045 4
[27] 0.50765374177488431506333 2 0.59956691099053782689765 1
[29] 0.73807436196544451423307 0 1.00000000000000000000000
```

Graph Centrality

```
# cut identifies the corresponding bin for each score
vals <- cut(eigenCent, bins, labels=FALSE,
            include.lowest=TRUE)
length(vals); head(vals)
```

```
[1] 132
```

```
[1] 10  5  2  5 19 18
```

```
# heat.colors creates a vector of n contiguous colors
my_col = heat.colors(length(bins))
head(my_col)
```

```
[1] "#FF0000" "#FF0C00" "#FF1700" "#FF2300" "#FF2E00" "#FF3
```

Graph Centrality

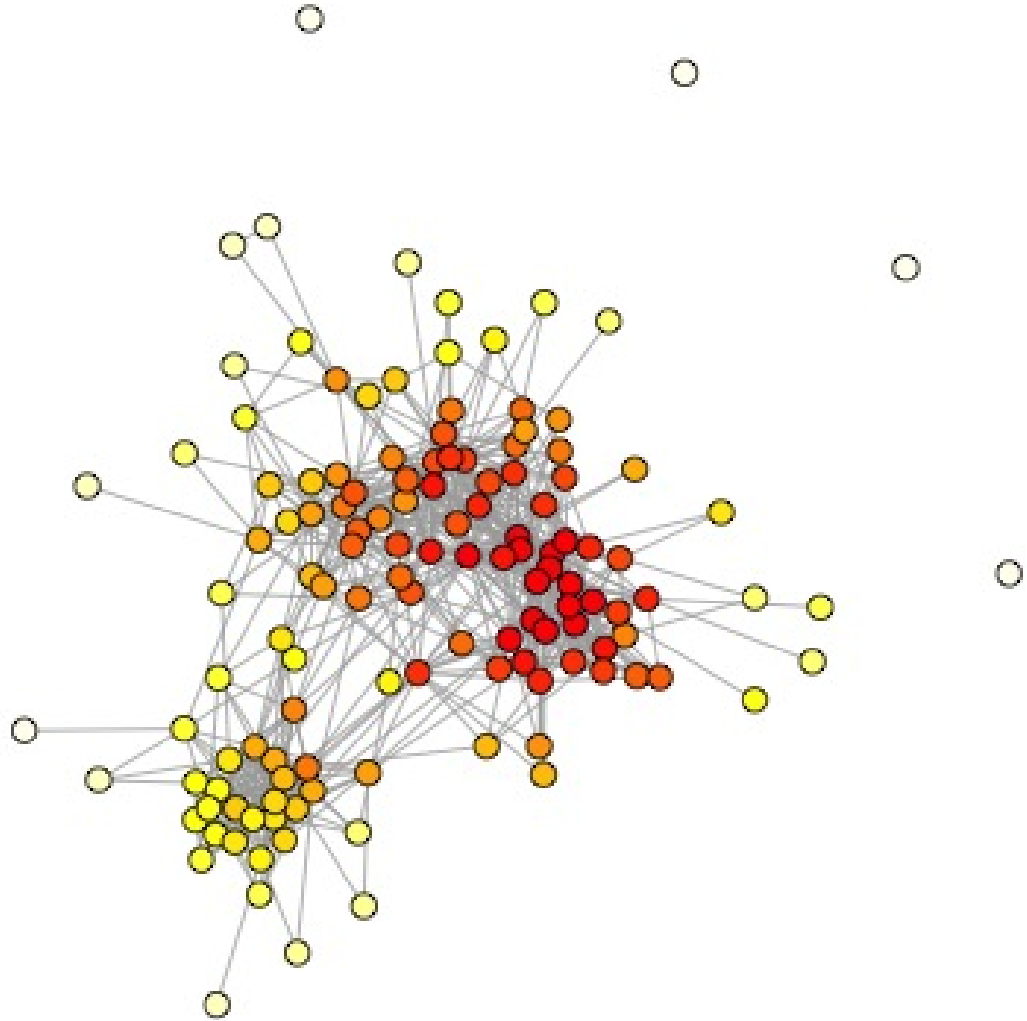
```
# now we assign the heatest colors (more red)
# to the highest scores
colorVals <- rev(my_col)[vals] #sort of subsetting
```

- To assign these colors (`colorVals`) to a plot, the easiest way is to attach a new element named color to the vertex set
- The plotting function will automatically pick up these colors and use them to shade the vertices

```
V(H)$color <- colorVals
```

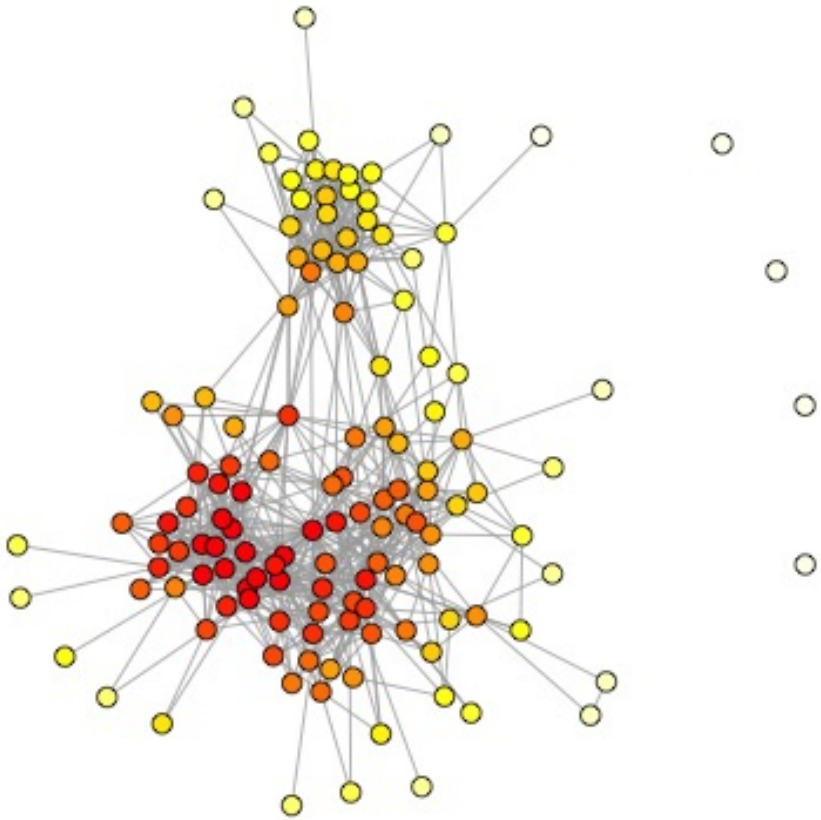
Graph Centrality

```
1  plot.igraph(H, vertex.label=NA, vertex.size=5)
```



Graph Centrality

More centrally located points are shaded darker with the isolated points the lightest



Graph Centrality

- The concept behind eigenvector centrality is that the most important vertices should be connected to many other well-connected vertices
- A slightly different notion of importance comes from identifying those vertices which connect disjoint parts of the graph
- The **betweenness centrality** measures this property of a graph by determining the shortest path between every pair of nodes and calculating how many of these run through each vertex

Graph Centrality

The **betweenness centrality** can also be easily calculated in the `igraph` library:

```
1 betweenCent <- betweenness(H)
2 sort(betweenCent,decreasing=TRUE)[1:10]
```

```
347US483  430US482  429US252  403US217  392US409
1964.8653  637.3044  591.0348  589.3720  519.2219
413US189 391US430  380US202 347US475  396US229
476.9320  388.6452  343.9670 333.3813  328.9941
```

```
1 cor(betweenCent,eigenCent)
```

```
[1] 0.5003805
```

Graph Centrality

- The highest betweenness score comes again from *Brown v. Board of Education*
- However, the next three cases are not in the top ten list from the eigenvector centrality scores
- Indeed, the correlation between the two measurements is only 0.5
- Clearly the betweenness score is significantly distinct from the eigenvector scores

Graph Centrality

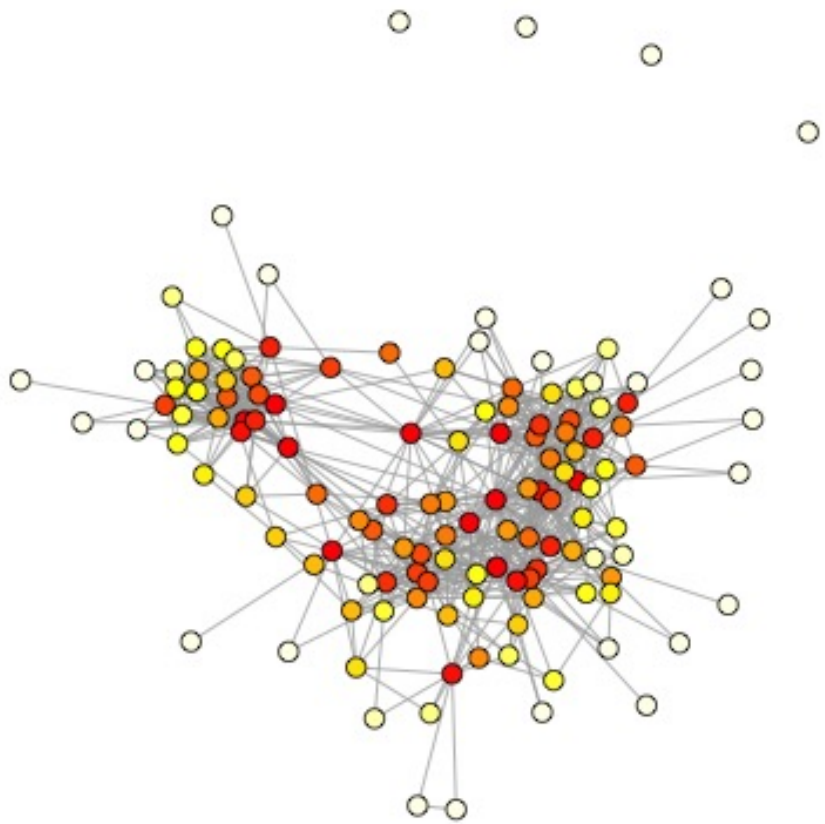
The betweenness score can be plotted on the graph using the same code as before:

```
bins <- unique(quantile(betweenCent,
                        seq(0,1,length.out=30)))
vals <- cut(betweenCent, bins, labels=FALSE,
            include.lowest=TRUE)
colorVals <- rev(heat.colors(length(bins)))[vals]
V(H)$color <- colorVals
```

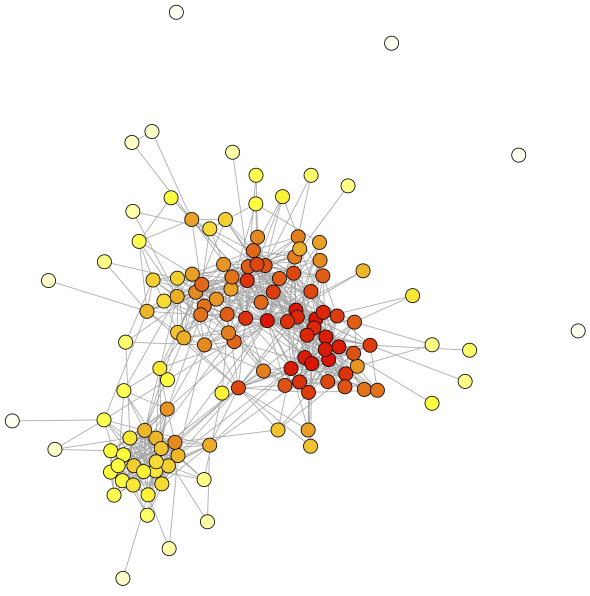
Graph Centrality

This new score does seem to favor vertices which connect disjoint parts of the graph:

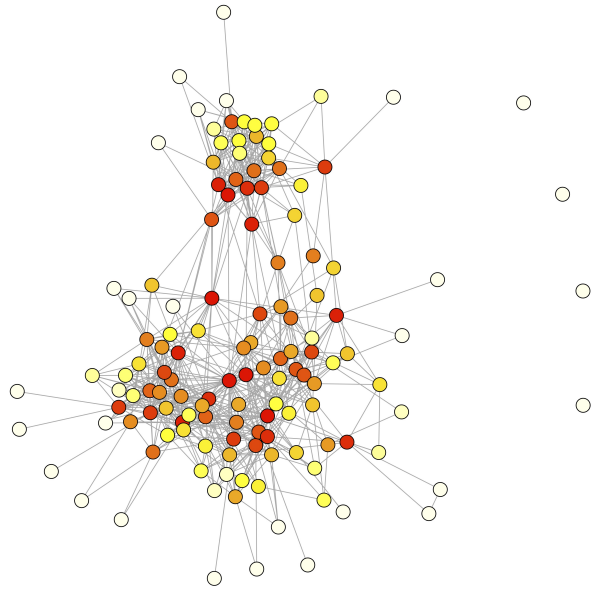
```
1  plot.igraph(H, vertex.label=NA, vertex.size=5)
```



Graph Centrality

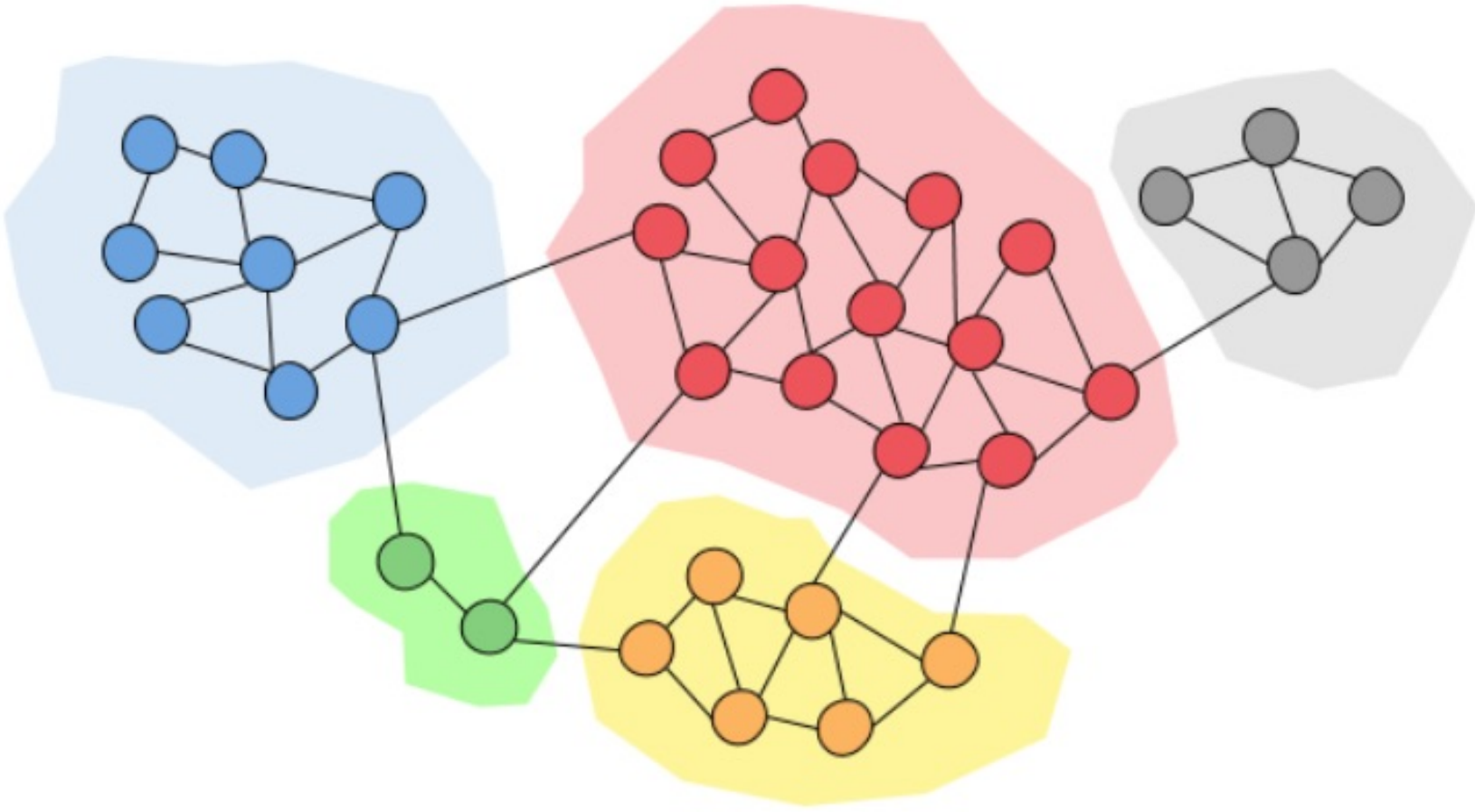


Eigenvector Centrality



Betweeness Centrality

Community Detection



Graph Communities

- There are many methods to split a graph into smaller clusters, known as **communities**
- Before, we calculated the number of minimal paths running through each vertex as a score for centrality
- The same process can be applied to the edges in a graph
  - The score indicatesthe degree to which the edge connects various sections of the graph
- Keeping removing edges with the highest levels of betweenness will eventually result in a disconnected graph
- The disconnected parts, which stayed together despite the removal of many edges, are then identified as the graph's communities

Graph Communities

- Calculating community structures requires only a simple function call
- However, creating a visualization with the result does take a little bit of data manipulation
- Take a look at the membership ids returned from running the betweenness community detection algorithm:

```
w <- edge.betweenness.community(H)
sort(table(w$membership))
```

```
 2  4  7  8  9 11 12 13 14 15 16 17 18 20 21 22 23 24 25
 1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
27 28 29 30 31 32 33 34 35 19 26  3  5  6  1 10
 1  1  1  1  1  1  1  1  1  2  2  4  6 22 29 39
```

Graph Communities

- There are three large communities with 22, 29, and 39 members and four smaller communities with no more than six members
- The other vertices are contained in 29 isolated singletons

2	4	7	8	9	11	12	13	14	15	16	17	18	20	21	22	23	24	25
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
27	28	29	30	31	32	33	34	35	19	26	3	5	6	1	10			
1	1	1	1	1	1	1	1	1	2	2	4	6	22	29	39			

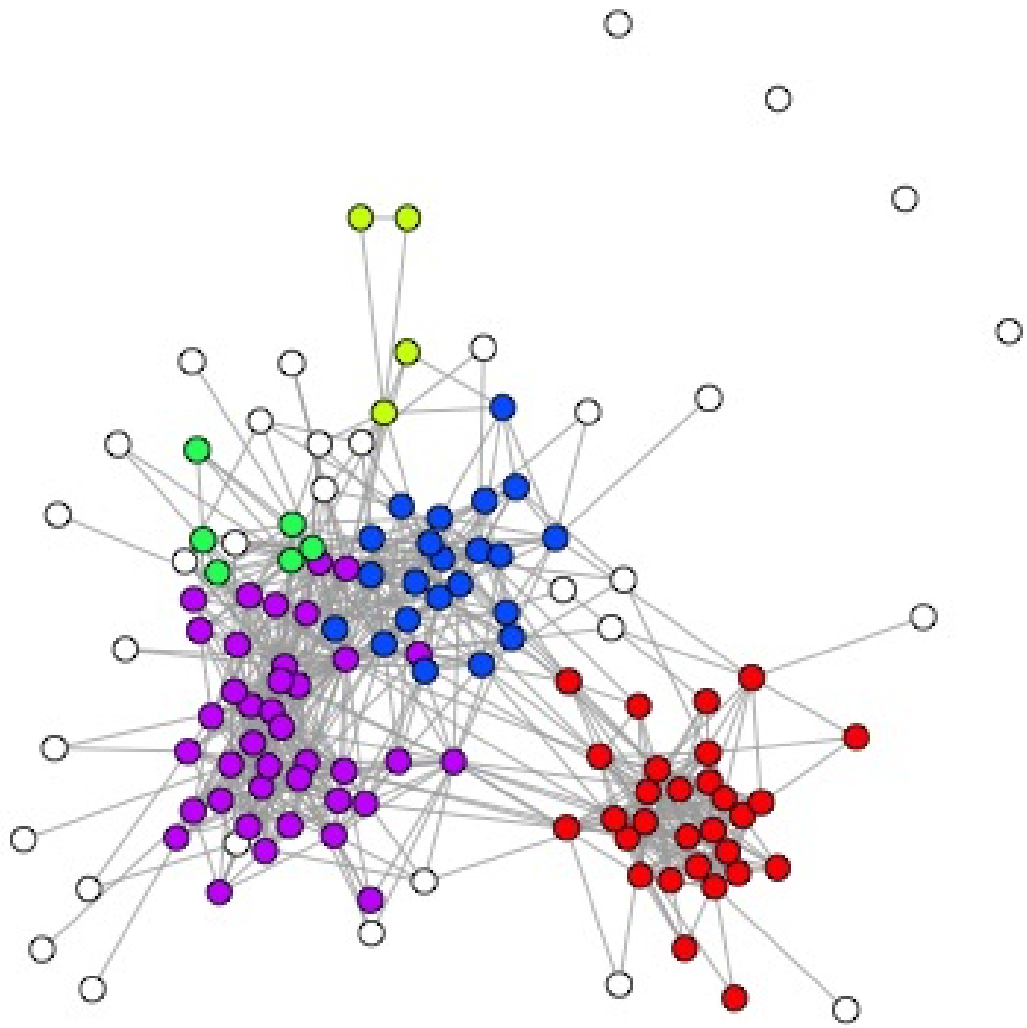
- We want to color the nontrivial communities (greater than 3) with different colors, while making the orphaned groups uncolored

Graph Communities

```
V(H)$color <- rep("white", length(w$membership))
keepTheseCommunities <- names(sizes(w))[sizes(w) > 3]
matchIndex <- match(w$membership, keepTheseCommunities)
# like %in%
colorVals <- rainbow(5)[matchIndex[!is.na(matchIndex)]]
V(H)$color[!is.na(matchIndex)] <- colorVals
```

Graph Communities

```
1  plot.igraph(H, vertex.label=NA, vertex.size=5)
```



Notes

This presentation is based on Chapter 6 of Arlnold and Tilton, *Humanities Data in R. Exploring Networks, Geospatial Data, Images, and Text.* © 2015 Springer

All code in the presentation is licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at http://www.apache.org/licenses/LICENSE-2.0 Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.